

MIT 212 — Programming Techniques

FINAL EXAMINATION MASTER PACKAGE

අවසාන විභාග ප්‍රධාන අධ්‍යයන ඇසුරුම — ද්විභාෂා (ඉංග්‍රීසි / සිංහල)

Revision sheets (Weeks 5, 11, 12) · Combined cheat sheet · Python syntax reference · 10 comparison tables · Glossary · 25 exam facts · 15 common mistakes · Night-before checklist · Full question bank with model answers

Master of Information Technology · Sabaragamuwa University of Sri Lanka

Based on the official lecture slides — Week 11 (Exception Handling) & Week 12 (Applications) by Kalinga Gunawardhana, plus Week 5 (Arrays & Structures) syllabus coverage.

1. One-Page Revision Sheet — Week 5: Arrays & Structures

සතිය 5 — අරා සහ ව්‍යුහ (Arrays & Structures) සංශෝධන පත්‍රිකාව

Week 5 goal: organise related data together — many values in one array/list, and many named fields in one structure (record).

සතිය 5 අරමුණ: සම්බන්ධිත දත්ත එකට සංවිධානය කිරීම — අගයන් රාශියක් එක list එකක ද, නම් කළ ක්ෂේත්‍ර (fields) රාශියක් එක structure එකක ද තබා ගැනීම.

Arrays / Lists | අරා / ලැයිස්තු

- A list stores an ordered collection of items under one name; each item is reached by an index starting at 0.
List එකක් යනු එක නමක් යටතේ අනුපිළිවෙලකට තබා ගත් අයිතම එකතුවකි; සෑම අයිතමයක්ම 0 සිට ආරම්භ වන index එකකින් ලබා ගැනේ.
- Negative indexes count from the end: marks[-1] is the last item.
සෘණ index අගයන් අගින් ගණන් කරයි: marks[-1] යනු අවසාන අයිතමයයි.
- A 2D array is a list of lists — grid[row][column] — used for tables such as marks per student per subject.
2D array එකක් යනු list of lists එකකි — grid[row][column] — ශිෂ්‍යයා අනුව විෂය අනුව ලකුණු වැනි වගු සඳහා යොදා ගැනේ.

```
marks = [72, 65, 80, 91]
print(marks[0])      # 72 (first)
print(marks[-1])     # 91 (last)
marks.append(55)     # add to end
grid = [[1, 2], [3, 4]]
print(grid[1][0])    # 3 (row 1, col 0)
```

Structures (records) | ව්‍යුහ (records)

- A structure keeps several related fields (id, name, grades) together in one unit — in Python, a dictionary or a dataclass.
Structure එකක් සම්බන්ධිත ක්ෂේත්‍ර කිහිපයක් (id, name, grades) එක ඒකකයක තබා ගනී — Python හි dictionary එකක් හෝ dataclass එකක් ලෙස.
- A list of structures is the standard pattern for many records: students = [{...}, {...}].
වාර්තා (records) රාශියක් සඳහා සම්මත රටාව list of structures වේ: students = [{...}, {...}].

```
student = {"id": "S001", "name": "Amara", "grades": [72, 65, 80]}
print(student["name"])      # Amara
student["grades"].append(90)

students = [
    {"id": "S001", "name": "Amara", "grades": [72, 65, 80]},
    {"id": "S002", "name": "Nadeesha", "grades": [45, 38, 50]},
]
for s in students:
    print(s["name"], sum(s["grades"]) / len(s["grades"]))
```

Why not a plain array of numbers? We would lose the link between a student and their own grades — structures keep related data together (exam favourite justification).

සරල සංඛ්‍යා array එකක් නොවන්නේ ඇයි? ශිෂ්‍යයා සහ ඔහුගේම ලකුණු අතර සම්බන්ධය නැති වේ — structures සම්බන්ධිත දත්ත එකට තබා ගනී (විභාගයට ප්‍රියතම හේතුව).

Key operations to memorise | මතක තබා ගත යුතු ප්‍රධාන මෙහෙයුම්

Operation මෙහෙයුම	List	Dictionary
Access / ප්‍රවේශය	x[0], x[-1]	d["key"], d.get("key")
Add / එකතු කිරීම	x.append(v), x.insert(i,v)	d["new"] = v
Remove / ඉවත් කිරීම	x.remove(v), x.pop(i)	del d["key"], d.pop("key")
Size / ප්‍රමාණය	len(x)	len(d)
Loop / ලූපය	for v in x:	for k, v in d.items():

Operation මෙහෙයුම	List	Dictionary
Membership / සාමාජිකත්වය	v in x	"key" in d

2. One-Page Revision Sheet — Week 11: Testing, Debugging & Exception Handling

සතිය 11 — පරීක්ෂණ, දෝෂ නිරාකරණය සහ ව්‍යතිරේක හැසිරවීම

Three kinds of bugs | දෝෂ වර්ග තුන

Type වර්ගය	What it means අර්ථය	Example උදාහරණය
Syntax error වාග් රීති දෝෂය	Broken grammar — caught before the program even runs. ව්‍යාකරණ බිඳී ඇත — වැඩසටහන ධාවනය වීමට පෙරම හඹුවේ.	missing colon / bracket
Runtime error ධාවන කාල දෝෂය	Valid syntax, but something fails while running. වාග් රීතිය නිවැරදි, නමුත් ධාවනය අතරතුර අසාර්ථක වේ.	10 / 0, open(missing)
Logic error තර්ක දෝෂය	Runs fine but gives the wrong answer — no crash, no warning. ක්‍රියාත්මක වේ, නමුත් වැරදි පිළිතුරක් දෙයි — crash හෝ අනතුරු ඇඟවීමක් නැත.	used len(data) instead of count

Debugging techniques (4) | දෝෂ නිරාකරණ ක්‍රම (4)

- Print debugging — add print() statements to see variable values at key points.
Print debugging — ප්‍රධාන ස්ථානවල විචල්‍ය අගයන් බැලීමට print() එකතු කරන්න.
- Trace by hand — walk through the code line by line on paper, tracking each variable.
අතින් trace කිරීම — කඩදාසියක කේතය පේළියෙන් පේළිය ගමන් කරමින් විචල්‍ය අගයන් සටහන් කරන්න.
- Use a debugger — step through code one line at a time and inspect state live.
Debugger එකක් — පේළියක් බැගින් ධාවනය කර වැඩසටහනේ තත්ත්වය සජීවීව පරීක්ෂා කරන්න.
- Rubber duck debugging — explaining the code line by line reveals the bug.
Rubber duck debugging — කේතය පේළියෙන් පේළිය කෙනෙකුට පැහැදිලි කිරීමේදී දෝෂය හෙළි වේ.

Reading a traceback | Traceback එකක් කියවීම

```
Traceback (most recent call last):  
  File "grades.py", line 12, in <module>  
    average = total / count  
ZeroDivisionError: division by zero
```

- File & line → exactly where the crash happened (start reading here). Error type → category of problem.
Message → what specifically went wrong.
File & line → බිඳවැටීම සිදු වූ ස්ථානය (කියවීම මෙතැනින් අරඹන්න). Error type → ගැටලුවේ කාණ්ඩය.
Message → නිශ්චිතව සිදු වූ වරද.

try / except / else / finally

```
try:  
    mark = int(input("Enter mark: "))  
except ValueError:  
    print("Invalid input.")  
else:  
    print("Mark accepted:", mark)  
finally:  
    print("Input attempt finished.")
```

Block කොටස	When it runs ධාවනය වන අවස්ථාව
try	Always attempted first. සැමවිටම පළමුව උත්සාහ කෙරේ.
except	Only if an error occurs inside try. try තුළ දෝෂයක් ඇති වූ විට පමණි.

Block කොටස	When it runs ධාවනය වන අවස්ථාව
else	Only if no error occurred. දෝෂයක් ඇති නොවූ විට පමණි.
finally	Always runs, no matter what. කුමක් වුවත් සැමවිටම ධාවනය වේ.

Order matters when catching multiple exceptions: list specific exceptions before generic ones. A bare except: catches everything — including bugs you didn't expect — so avoid it unless you re-raise or log the error.
ව්‍යතිරේක කිහිපයක් අල්ලන විට අනුපිළිවෙල වැදගත්: නිශ්චිත ව්‍යතිරේක සාමාන්‍ය ඒවාට පෙර ලියන්න.
හිස් except: සියල්ල අල්ලයි — බලාපොරොත්තු නොවූ දෝෂ ද ඇතුළුව — එබැවින් එය වළක්වන්න.

Common built-in exceptions + raise | සාමාන්‍ය ව්‍යතිරේක සහ raise

Exception ව්‍යතිරේකය	Triggered by ඇති කරන්නේ
ValueError	int("abc")
TypeError	"5" + 5
ZeroDivisionError	10 / 0
IndexError	marks[10] # only 6 items
KeyError	students["S099"]
FileNotFoundError	open("missing.csv")

```
def set_mark(student_id, mark):
    if not 0 <= mark <= 100:
        raise ValueError("Mark must be between 0 and 100")
    students[student_id] = mark
```

raise මගින් ඔබේම කේතයට යමක් වැරදි බව සංඥා කළ හැක — භාෂාව නොදන්නා නීති (උදා: වලංගු ලකුණු පරාසය) බලාත්මක කිරීමට ප්‍රයෝජනවත්. Caller ට එය සාමාන්‍ය try/except එකකින් අල්ලිය හැක.

3. One-Page Revision Sheet — Week 12: Applications, Case Studies & Capstone

සතිය 12 — යෙදුම්, සිද්ධි අධ්‍යයන සහ Capstone ව්‍යාපෘතිය

The six-step problem-solving process | ගැටලු විසඳීමේ පියවර හය

#	Step පියවර	Question / output ප්‍රශ්නය / ප්‍රතිදානය
1	Understand අවබෝධ කරගන්න	What is being asked?
2	Decompose කුඩා කොටස්වලට බෙදන්න	Split into small tasks
3	Choose structures ව්‍යුහ තෝරන්න	Which data structure fits?
4	Design algorithm ඇල්ගොරිතමය සැලසුම් කරන්න	Pseudocode / flowchart
5	Implement ක්‍රියාත්මක කරන්න	Write the code
6	Test & refine පරීක්ෂා කර වැඩිදියුණු කරන්න	Debug, then improve

Which data structure do I need? | අවශ්‍ය දත්ත ව්‍යුහය කුමක්ද?

Need අවශ්‍යතාවය	Structure ව්‍යුහය
Serve items in arrival order ඇමිණි අනුපිළිවෙලින් සේවය	Queue — FIFO (Week 8)
Undo the most recent action first අවසාන ක්‍රියාව මුලින්ම අහෝසි කිරීම	Stack — LIFO (Week 8)
Fast lookup by a sorted key වර්ග කළ යතුරකින් වේගවත් සෙවීම	Tree / BST — log-time search (Week 9)
Records must survive after the program closes වැඩසටහන වැසුණු පසුත් වාර්තා පැවතිය යුතුය	File organisation — sequential / random / linked / inverted (Week 10)

BST vs plain list: searching a plain list of 10,000 titles checks about 5,000 on average; a balanced BST halves the search each step — about 14 comparisons for the same 10,000 titles.

BST සහ සරල list: මාතෘකා 10,000ක සරල list එකක සෙවීමේදී සාමාන්‍යයෙන් 5,000ක් පරීක්ෂා වේ; සමතුලිත BST එකක් සෑම පියවරකදීම සෙවුම අඩකින් අඩු කරයි — එම 10,000 සඳහා සැසඳීම් 14ක් පමණ.

The three case studies — techniques combined | සිද්ධි අධ්‍යයන තුන — ඒකාබද්ධ කළ ක්‍රම

Case study සිද්ධි අධ්‍යයනය	Techniques ක්‍රම	Key idea ප්‍රධාන අදහස
1. Student Grade Management	Structures · Functions · Exceptions · Branching	One bad record is skipped safely; the whole class report still finishes. නරක වාර්තාවක් ආරක්ෂිතව මග හැරේ; සම්පූර්ණ වාර්තාව අවසන් වේ.
2. Library Directory & Search	BST · Linked lists · Files · Exceptions	Sequential file rebuilds the fast-search BST at startup; missing file starts an empty catalog instead of crashing. Sequential file එකෙන් BST එක නැවත ගොඩනැගේ; file එක නැති විට crash නොවී හිස් catalog එකක් අරඹයි.
3. Print Job Queue Simulation	Queue · Stack · Recursion · Exceptions	Queue = who prints next (FIFO); Stack = undo last submission (LIFO); recursion totals nested batch pages.

Case study සිද්ධි අධ්‍යයනය	Techniques ක්‍රම	Key idea ප්‍රධාන අදහස
		Queue = මිලිතට මුද්‍රණය (FIFO); Stack = අවසාන ඉදිරිපත් කිරීම අහෝසි කිරීම (LIFO); recursion මගින් nested batch පිටු එකතුව.

Key code patterns from Week 12 | සතිය 12 හි ප්‍රධාන කේත රටා

```
# Safe per-record processing (Case Study 1)
def process_student(record):
    try:
        avg = average(record["grades"])
        status = "AT RISK" if is_at_risk(avg) else "OK"
        return f"{record['name']}: avg={avg:.1f} [{status}]"
    except (KeyError, TypeError) as e:
        return f"Skipped {record.get('id','?')} - bad record ({e})"

# Queue + undo stack fed by one event (Case Study 3)
def submit_job(job):
    print_queue.append(job)    # queue: printing order
    undo_stack.append(job)    # stack: undo button

# Recursion for nested batches
def total_pages(job):
    if "sub_jobs" not in job:    # base case
        return job["pages"]
    return sum(total_pages(j) for j in job["sub_jobs"])
```

Capstone: Library Management System | Capstone: පුස්තකාල කළමනාකරණ පද්ධතිය

- Four teams, four modules, one shared system: Borrow & Return, Search Engine, Waitlist Manager, Reports & Validation.
කණ්ඩායම් හතරක්, module හතරක්, එක බෙදාගත් පද්ධතියක්: Borrow & Return, Search Engine, Waitlist Manager, Reports & Validation.
- Every module reads and writes the same book_id / student_id keys — that shared data contract is what makes integration possible.
සෑම module එකක්ම එකම book_id / student_id යතුරු කියවා ලියයි — එම බෙදාගත් data contract එක නිසාම ඒකාබද්ධ කිරීම හැකි වේ.
- Integration rule: agree on key formats BEFORE building — mismatched keys are the #1 reason integrations fail.
ඒකාබද්ධතා නීතිය: ගොඩනැගීමට පෙර key ආකෘති එකඟ වන්න — නොගැලපෙන keys යනු ඒකාබද්ධතා අසාර්ථක වීමේ #1 හේතුවයි.

4. Combined Revision Sheet — Weeks 11 & 12 Together

සති 11 සහ 12 ඒකාබද්ධ සංශෝධන පත්‍රිකාව

How the two weeks connect: Week 11 gives you the safety tools (testing, tracebacks, try/except); Week 12 applies them inside every real system — every case study wraps risky steps (file loads, user input, lookups) in exception handling.

සති දෙක සම්බන්ධ වන ආකාරය: සතිය 11 ආරක්ෂක මෙවලම් ලබා දෙයි (පරීක්ෂණ, traceback, try/except); සතිය 12 ඒවා සෑම සැබෑ පද්ධතියක් තුළම යොදයි — සෑම සිද්ධි අධ්‍යයනයක්ම අවදානම් පියවර (file කියවීම්, user input, lookups) exception handling තුළ ඔතයි.

Concept සංකල්පය	Week 11 (tool) සතිය 11 (මෙවලම)	Week 12 (application) සතිය 12 (යෙදුම)
Errors	Syntax / runtime / logic; read the traceback bottom-up.	Hit an error? → Debug & Fix loop until the program behaves.
try/except	Catch ValueError, KeyError, TypeError...; else on success; finally always.	Grade report skips a bad record; catalog loader survives a missing file (FileNotFoundError).
raise	raise ValueError("Mark must be between 0 and 100")	borrow() raises ValueError if the book is already on loan.
Debugging	print / trace by hand / debugger / rubber duck.	Team walkthroughs: explain one bug you hit and how you found it.
Pattern	Wrap only risky lines; catch specific exceptions first.	Every case study: pick the right structure, write small functions, wrap risky steps in exceptions.

Exam-ready sequence | විභාගයට සූදානම් අනුපිළිවෙල

- 1) Classify the error type → 2) locate it via traceback → 3) protect the risky line with try/except → 4) catch the specific exception → 5) use else/finally where the question asks → 6) validate with raise for business rules.
 - 1) දෝෂ වර්ගය හඳුනාගන්න → 2) traceback එකෙන් ස්ථානය සොයන්න → 3) අවදානම් පේළිය try/except වලින් ආරක්ෂා කරන්න → 4) නිශ්චිත exception එක අල්ලන්න → 5) ප්‍රශ්නය ඉල්ලන විට else/finally යොදන්න → 6) ව්‍යාපාර නීති සඳහා raise භාවිතා කරන්න.

5. Combined Final-Exam Cheat Sheet (Whole Course)

සම්පූර්ණ පාඨමාලාවේ අවසාන විභාග Cheat Sheet

Topic (Week) මාතෘකාව (සතිය)	One-line essence එක පේළියේ සාරය	Must-know අනිවාර්යයෙන් දැනගත යුතු
Variables & Data (1)	Store and manipulate information. තොරතුරු ගබඩා කර හසුරුවයි.	int, float, str, bool; type conversion
Flow Control (2)	Make decisions and repeat steps. නිරණ ගනී, පියවර පුනරාවර්තනය කරයි.	if/elif/else, for, while, break/continue
Functions (2–3)	Break code into reusable pieces. නැවත භාවිත කළ හැකි කොටස්.	def, parameters, return, default args
Arrays & Structures (5)	Organise related data together. සම්බන්ධිත දත්ත එකට සංවිධානය.	list, dict, list of dicts, 2D lists
Pointers / References (6)	Understand how data connects. දත්ත සම්බන්ධ වන ආකාරය.	assignment shares a reference; copy() for a real copy
Recursion (7)	Solve problems by breaking them down. ගැටලුව කඩා විසඳීම.	base case + recursive case
Data Structures (8–9)	Choose the right tool for the job. නිවැරදි මෙවලම තෝරන්න.	stack LIFO, queue FIFO, linked list, BST
File Organisation (10)	Make data survive and stay searchable. දත්ත රැඳී පවතින සෙවිය හැකි බවට.	sequential, random, linked, inverted
Exception Handling (11)	Build systems that don't crash. Crash නොවන පද්ධති.	try/except/else/finally, raise
Applications (12)	Combine it all into real systems. සියල්ල සැබෑ පද්ධතිවලට එකාබද්ධ කිරීම.	6-step process; 3 case studies; capstone

Every case study followed the same shape: pick the right structure → write small functions → wrap risky steps in exceptions. Quote this pattern in essay answers.

සෑම සිද්ධි අධ්‍යයනයක්ම එකම හැඩය අනුගමනය කළේය: නිවැරදි ව්‍යුහය තෝරන්න → කුඩා ශ්‍රිත ලියන්න → අවදානම් පියවර exceptions වලින් ඔතන්න. රචනා පිළිතුරුවලදී මෙම රටාව සඳහන් කරන්න.

6. Python Syntax Reference Sheet

Python වාග් රීති (Syntax) යොමු පත්‍රිකාව

Variables, types, input/output | විචල්‍ය, වර්ග, ආදාන/ප්‍රතිදාන

```
name = "Amara"          # str
age = 24                 # int
gpa = 3.72               # float
passed = True            # bool
mark = int(input("Mark: ")) # read + convert
print(f"{name}: avg={gpa:.1f}") # f-string, 1 decimal
```

Selection | තේරීම (Selection)

```
if avg >= 75:
    grade = "A"
elif avg >= 40:
    grade = "Pass"
else:
    grade = "AT RISK"
status = "AT RISK" if avg < 40 else "OK" # ternary
```

Loops | ලූප්

```
for i in range(5):      # 0,1,2,3,4
    print(i)
for s in students:      # over a list
    print(s["name"])
while count > 0:
    count -= 1
# break = leave loop now, continue = skip to next turn
```

Functions | ශ්‍රිත

```
def average(grades):
    if len(grades) == 0:
        return 0
    return sum(grades) / len(grades)

def is_at_risk(avg, cutoff=40): # default parameter
    return avg < cutoff
```

Lists, dicts, strings | List, dict, string

```
marks = [72, 65, 80]
marks[0]; marks[-1]; marks[0:2] # index & slice
marks.append(90); marks.remove(65); len(marks)
d = {"id": "S001"}; d["name"] = "Amara"
d.get("grades", []) # safe access
line = "Dune|Herbert"
title, author = line.strip().split("|")
```

Exceptions | ව්‍යතිරේක

```
try:
    risky()
except (KeyError, TypeError) as e:
    print("Bad record:", e)
except ZeroDivisionError:
    print("No marks to average.")
else:
    print("Success")
finally:
    print("Done")
raise ValueError("Mark must be between 0 and 100")
```

Files | ගොනු

```
try:
```

```
with open("library.txt", "r") as f:
    for line in f:
        process(line)
except FileNotFoundError:
    print("No catalog file found - starting empty")
with open("borrow_log.txt", "a") as f: # append
    f.write(f"{book_id}|{student_id}\n")
```

Stack, queue, recursion | [Stack, queue, recursion](#)

```
stack = []; stack.append(x); top = stack.pop() # LIFO
queue = []; queue.append(x); first = queue.pop(0) # FIFO

def total_pages(job):
    if "sub_jobs" not in job: # base case
        return job["pages"]
    return sum(total_pages(j) for j in job["sub_jobs"])
```

7. Comparison Tables

සංසන්දන වගු

7.1 Algorithm vs Program | ඇල්ගොරිතමය සහ වැඩසටහන

Aspect අංශය	Algorithm ඇල්ගොරිතමය	Program වැඩසටහන
Definition / අර්ථය	A step-by-step plan to solve a problem, independent of any language. ඕනෑම භාෂාවකින් ස්වාධීන, ගැටලුවක් විසඳීමේ පියවරෙන් පියවර සැලැස්මකි.	The algorithm implemented in a programming language so a computer can run it. පරිගණකයකට ධාවනය කළ හැකි ලෙස ක්‍රමලේඛ භාෂාවකින් ලියූ ඇල්ගොරිතමයයි.
Form / ස්වරූපය	Pseudocode, flowchart, plain steps	Source code (e.g., Python file)
Runs? / ධාවනය?	Cannot be executed directly	Executable by a computer
Example / උදාහරණය	"Add all grades, divide by count"	<code>return sum(grades)/len(grades)</code>

7.2 Compiler vs Interpreter | සම්පාදකය සහ පරිවර්තකය

Aspect අංශය	Compiler සම්පාදකය	Interpreter පරිවර්තකය
Translation / පරිවර්තනය	Translates the whole program to machine code before running. ධාවනයට පෙර සම්පූර්ණ වැඩසටහනම යන්ත්‍ර කේතයට පරිවර්තනය කරයි.	Translates and executes line by line at run time. ධාවන කාලයේදී පේළියෙන් පේළිය පරිවර්තනය කර ක්‍රියාත්මක කරයි.
Errors / දෝෂ	All syntax errors reported together before execution	Stops at the first error it meets while running
Speed / වේගය	Faster execution after compilation	Slower execution; faster to start testing
Output / ප්‍රතිදානය	Separate executable file	No separate executable
Examples / උදාහරණ	C, C++	Python (mainly interpreted)

7.3 Syntax vs Runtime vs Logic error | වාග් රීති / ධාවන කාල / තර්ක දෝෂ

Aspect අංශය	Syntax error	Runtime error	Logic error
When detected / හඳුනා විට	Before the program runs	While running	Never automatically — output is just wrong
Cause / හේතුව	Broken grammar (missing colon, bracket)	Valid code meets an impossible situation (10/0, missing file)	Wrong reasoning by the programmer
Program crashes? / crash වේද?	Never starts	Crashes (unless caught with try/except)	Runs fine, wrong answer
Example / උදාහරණය	<code>if x > 5 print(x)</code>	<code>total / count # count==0</code>	<code>total/len(data) instead of /count</code>
Fix tool / විසඳුම	Read the SyntaxError message	Traceback + try/except	Testing, tracing, print debugging

7.4 if vs if-else vs if-elif-else

Form ආකෘතිය	Paths මාර්ග	Use when භාවිත කරන විට	Example
if	1 optional action	Do something only when a condition is true; otherwise do nothing. කොන්දේසිය සත්‍ය විට පමණක් ක්‍රියා කරන්න.	<code>if avg < 40: flag()</code>
if-else	Exactly 2 paths	Two mutually exclusive outcomes — one always runs.	"AT RISK" if avg<40 else "OK"

Form ආකෘතිය	Paths මාර්ග	Use when භාවිත කරන විට	Example
		අන්‍යෝන්‍ය වශයෙන් බැහැර ප්‍රතිඵල දෙකක් — එකක් සැමවිට ධාවනය වේ.	
if-elif-else	3 or more paths	Several exclusive ranges/cases checked top to bottom; first true branch wins. පරාස කිහිපයක් ඉහළ සිට පහළට පරීක්ෂා වේ; පළමු සත්‍ය ශාඛාව ජය ගනී.	grade bands A/B/C/F

7.5 for loop vs while loop | for ලූපය සහ while ලූපය

Aspect අංශය	for loop	while loop
Best when / හොඳම විට	The number of repetitions or the collection is known in advance. පුනරාවර්තන ගණන හෝ එකතුව කලින් දන්නා විට.	Repeats until a condition changes — count unknown in advance. කොන්දේසියක් වෙනස් වන තුරු — ගණන කලින් නොදන්නා විට.
Driven by / පාලනය	A sequence: range(), list, file lines	A boolean condition
Risk / අවදානම	Low — ends when the sequence ends	Infinite loop if the condition never becomes False
Example / උදාහරණය	for s in students: ...	while attempts < 3: ...

7.6 print() vs return | print() සහ return

Aspect අංශය	print()	return
Purpose / අරමුණ	Displays a value to the user on screen. අගයක් තිරයේ පෙන්වයි.	Sends a value back to the caller for further use. අගයක් caller වෙත ආපසු යවයි — නවදුරටත් භාවිතයට.
Where / කොතැනද	Anywhere	Only inside a function; ends the function immediately
Value reusable? / නැවත භාවිතය	No — only visible, not stored	Yes — can be assigned: x = f()
Exam trap / උගුල	print(f()) shows None if f() only prints	code after return never runs

7.7 Parameter vs Argument | පරාමිතිය සහ තර්කය

Aspect අංශය	Parameter පරාමිතිය	Argument තර්කය
What / කුමක්ද	The variable name in the function definition. ශ්‍රිත අර්ථදැක්වීමේ ඇති විචල්‍ය නාමයයි.	The actual value supplied when the function is called. ශ්‍රිතය කැඳවන විට සපයන සැබෑ අගයයි.
Where / කොතැනද	def is_at_risk(avg, cutoff=40):	is_at_risk(38.7, 45)
Count fixed? / ස්ථිරද	Fixed by the definition (defaults optional)	Must match parameters (defaults can be omitted)

7.8 Primitive vs Non-primitive data | ප්‍රාථමික සහ ප්‍රාථමික නොවන දත්ත

Aspect අංශය	Primitive ප්‍රාථමික	Non-primitive ප්‍රාථමික නොවන
Holds / දරන්නේ	A single simple value. තනි සරල අගයක්.	A collection or structure of values. අගයන් එකතුවක් හෝ ව්‍යුහයක්.
Examples / උදාහරණ	int, float, bool, (str)	list, dict, tuple, set, objects
Divisible? / බෙදිය හැකිද	Not broken into smaller parts	Made of elements you can index / iterate

Aspect අංශය	Primitive ප්‍රාථමික	Non-primitive ප්‍රාථමික නොවන
In this course / පාඨමාලාවේදී	marks = 72	students = [{"id": ...}]

7.9 Indexing vs Slicing | සුවිගත කිරීම සහ පෙති කැපීම

Aspect අංශය	Indexing	Slicing
Returns / ලබා දෙන්නේ	One item. එක් අයිතමයක්.	A new list/string containing a range of items. පරාසයක අයිතම සහිත නව list/string එකක්.
Syntax / රීතිය	marks[2], marks[-1]	marks[1:4], marks[:3], marks[:2]
End bound / අවසන් සීමාව	Exact position (0-based)	Stop index is EXCLUDED: [1:4] gives positions 1,2,3
Error risk / දෝෂ	IndexError if out of range	No error — out-of-range slice just shortens

7.10 Mutable vs Immutable | වෙනස් කළ හැකි සහ නොහැකි

Aspect අංශය	Mutable වෙනස් කළ හැකි	Immutable වෙනස් කළ නොහැකි
Meaning / අර්ථය	Contents can change in place after creation. සෑදූ පසු අන්තර්ගතය එතැනම වෙනස් කළ හැක.	Cannot change; any 'change' creates a new object. වෙනස් කළ නොහැක; 'වෙනසක්' නව object එකක් සාදයි.
Types / වර්ග	list, dict, set	int, float, str, bool, tuple
Reference effect / යොමු බලපෑම	b = a; b.append(9) changes a too!	b = a; b = b + 1 leaves a unchanged
Exam link / විභාග සම්බන්ධය	Week 6 pointers/references: assignment shares, copy() duplicates	s.upper() returns a NEW string

8. Glossary of Essential Terminology

අත්‍යවශ්‍ය පාරිභාෂික ශබ්දමාලාව

Term අදය	සිංහල	Definition අර්ථ දැක්වීම
Algorithm	ඇල්ගොරිතමය	A finite, ordered set of steps that solves a problem.
Program	වැඩසටහන	An algorithm written in a programming language for a computer to execute.
Bug	දෝෂය	An error that makes a program crash, misbehave, or produce the wrong result.
Debugging	දෝෂ නිරාකරණය	The systematic process of finding and fixing bugs.
Traceback	ට්‍රේස්බැක්	Python's crash report: file & line, error type, and message.
Exception	ව්‍යතිරේකය	An event during execution that disrupts the normal flow of instructions.
try / except	—	Statement that attempts risky code and catches errors so the program continues.
else (in try)	—	Block that runs only when no error occurred in the try block.
finally	—	Block that always runs, whether or not an error occurred.
raise	—	Keyword that signals an error from your own code, enforcing your own rules.
Syntax error	වාග් රීති දෝෂය	Broken grammar; caught before the program even runs.
Runtime error	ධාවන කාල දෝෂය	Valid syntax that fails while running (e.g., division by zero).
Logic error	තර්ක දෝෂය	Program runs but gives the wrong answer — no crash, no warning.
Variable	චිචල්‍යය	A named location that stores a value.
Function	ශ්‍රිතය	A named, reusable block of code that may take parameters and return a value.
Parameter	පරාමිතිය	Variable named in a function definition.
Argument	තර්කය	Actual value passed to a function when calling it.
Array / List	අරාව / ලැයිස්තුව	Ordered collection of items accessed by 0-based index.
Structure / Record	ව්‍යුහය / වාර්තාව	Several related fields kept together (Python dict / dataclass).
Dictionary	ශබ්දකෝෂ ව්‍යුහය	Key → value mapping; lookup by key, e.g., students["S001"].
Index	සූචිය	Position number of an item; first item is index 0.
Slice	පෙත්ත	A sub-range of a sequence, e.g., marks[1:4] (stop excluded).
Mutable	වෙනස් කළ හැකි	Object whose contents can change in place (list, dict).
Immutable	වෙනස් කළ නොහැකි	Object that cannot change after creation (int, str, tuple).
Reference	යොමුව	A link to an object; assignment copies the reference, not the data.
Recursion	ප්‍රත්‍යාවර්තනය	A function calling itself with a base case and a recursive case.
Base case	පාදක අවස්ථාව	The condition where recursion stops without another call.
Stack	ස්ටැක් (අවටිය)	LIFO structure — last in, first out; used for undo.
Queue	පෝලීම	FIFO structure — first in, first out; used for arrival order.

Term පදය	සිංහල	Definition අර්ථ දැක්වීම
Linked list	සම්බන්ධිත ලැයිස්තුව	Nodes connected by references; easy insert/remove, linear search.
Binary Search Tree (BST)	ද්විමය සෙවුම් වෘක්ෂය	Sorted tree; halves the search space each comparison (log-time).
Sequential file	අනුක්‍රමික ගොනුව	One record per line, read top to bottom.
Pseudocode	ව්‍යාජ කේතය	Language-neutral, structured description of an algorithm.
Flowchart	ගැලීම් සටහන	Diagram of an algorithm using standard symbols.
Compiler	සම්පාදකය	Translates a whole program into machine code before running.
Interpreter	පරිවර්තකය	Translates and executes code line by line.
Decomposition	විසංයෝජනය	Splitting a big problem into small, solvable tasks.
Data contract	දත්ත ගිවිසුම	Shared key formats (book_id, student_id) that modules agree on before integrating.
Integration	ඒකාබද්ධ කිරීම	Joining separately built modules into one working system.
Validation	වලංගුකරණය	Checking input obeys the rules before using it (often with raise / try).

9. Twenty-Five High-Priority Examination Facts

විභාගයට ඉහළ ප්‍රමුඛතා කරුණු 25

1. A bug is an error that makes a program crash, misbehave, or produce the wrong result.

Bug = crash වීම, වැරදි හැසිරීම හෝ වැරදි ප්‍රතිඵලයක් ඇති කරන දෝෂයකි.

2. Syntax errors are caught BEFORE the program runs; runtime errors happen WHILE running; logic errors never crash — the answer is simply wrong.

Syntax දෝෂ ධාවනයට පෙරම; runtime දෝෂ ධාවනය අතරතුර; logic දෝෂ කිසිදා crash නොවේ — පිළිතුර වැරදියි පමණි.

3. A traceback shows three things: file & line, error type, and message — start reading at the last line.

Traceback එක file & line, error type, message පෙන්වයි — අවසාන පේළියෙන් කියවීම අරඹන්න.

4. try runs first; except runs only on error; else runs only on success; finally always runs.

try පළමුව; except දෝෂයකදී පමණි; else සාර්ථක වූ විට පමණි; finally සැමවිටම.

5. List specific exceptions before generic ones; avoid a bare except: because it hides unexpected bugs.

නිශ්චිත exceptions සාමාන්‍ය ඒවාට පෙර; හිස් except: වළක්වන්න.

6. `int("abc")` → `ValueError`; `"5"+5` → `TypeError`; `10/0` → `ZeroDivisionError`; `marks[10]` on a 6-item list → `IndexError`; `students["S099"]` missing → `KeyError`; `open("missing.csv")` → `FileNotFoundError`.

සය ප්‍රධාන ව්‍යතිරේක සහ ඒවා ඇති කරන කේත මතක තබාගන්න.

7. `raise` lets your own code signal an error — used to enforce rules the language doesn't know, like $0 \leq \text{mark} \leq 100$.

`raise` මගින් ඔබේම නීති (උදා: $0 \leq \text{mark} \leq 100$) බලාත්මක කළ හැක.

8. `except (TypeError, ValueError):` catches either exception with one handler.

එක handler එකකින් ව්‍යතිරේක දෙකක් ඇල්ලීමට tuple එකක් යොදන්න.

9. Adding `None` to a number raises `TypeError` — that is why the grade calculator skipped `None` marks and counted only valid ones.

`None` එකතු කිරීම `TypeError` දෙයි — grade calculator එක `None` මග හැරයේ ඒ නිසාය.

10. The four debugging techniques: print debugging, trace by hand, use a debugger, rubber duck debugging.

දෝෂ නිරාකරණ ක්‍රම හතර මතක තබාගන්න.

11. The six-step problem-solving process: Understand → Decompose → Choose structures → Design algorithm → Implement → Test & refine.

ගැටලු විසඳීමේ පියවර හය අනුපිළිවෙලින්ම මතක තබාගන්න.

12. Queue = FIFO = serve items in arrival order; Stack = LIFO = undo the most recent action first.

Queue = FIFO; Stack = LIFO.

13. In a Python list: queue → `append()` + `pop(0)`; stack → `append()` + `pop()`.

List එකකින්: queue → `append()`+`pop(0)`; stack → `append()`+`pop()`.

14. A balanced BST halves the search space at each step: ~14 comparisons for 10,000 titles vs ~5,000 on average for a plain list.

සමතුලිත BST: 10,000 සඳහා ~14 සැසඳීම්; සරල list: ~5,000.

15. In a BST, smaller titles go left, larger (or equal) go right — "Dune" < "Moby Dick" so it goes left.

BST හි කුඩා අගයන් වමට, විශාල ඒවා දකුණට.

16. Recursion needs a base case (stop) and a recursive case (self-call on smaller input) — `total_pages()` returns `job["pages"]` as the base case.

Recursion ට base case සහ recursive case අවශ්‍යයි.

17. Files make records survive after the program closes; a sequential file holds one record per line, read top to bottom.

ගොනු මගින් වාර්තා වැඩසටහන වැසුණු පසුත් පවතී.

18. Catching `FileNotFoundError` when loading lets the program start with an empty catalog instead of crashing.

`FileNotFoundError` ඇල්ලීමෙන් crash නොවී හිස් catalog එකකින් ආරම්භ විය හැක.

19. A structure keeps related fields (id, name, grades) together; a list of structures holds many records.

Structure = සම්බන්ධිත ක්ෂේත්‍ර එකට; list of structures = වාර්තා රාශියක්.

20. Slicing excludes the stop index: marks[1:4] returns positions 1, 2, 3 only.

Slice එකේ stop index ඇතුළත් නොවේ.

21. Assignment copies a reference, not the data: b = a then b.append(9) also changes a (mutable objects).

Assignment යනු reference පිටපතකි — mutable object වලදී දෙකම වෙනස් වේ.

22. print() displays a value; return sends it back to the caller and ends the function immediately.

print() පෙන්වයි; return ආපසු යවා ශ්‍රිතය අවසන් කරයි.

23. submit_job() feeds both the queue (printing order) and the stack (undo) — same event, two views of it.

එකම සිදුවීම ව්‍යුහ දෙකකට — queue (අනුපිළිවෙල) සහ stack (undo).

24. In the capstone, every module reads/writes the same book_id / student_id keys — the shared data contract makes integration possible; mismatched keys are the #1 integration failure.

බෙදාගත් data contract එකම ඒකාබද්ධතාවය හැකි කරයි; නොගැලපෙන keys #1 අසාර්ථක හේතුවයි.

25. Every case study followed the same shape: pick the right structure, write small functions, wrap risky steps in exceptions.

සෑම සිද්ධි අධ්‍යයනයකම හැඩය: නිවැරදි ව්‍යුහය → කුඩා ශ්‍රිත → අවදානම් පියවර exceptions වලින්.

10. Fifteen Common Programming Mistakes & Corrections

සාමාන්‍ය ක්‍රමලේඛන වැරදි 15 සහ නිවැරදි කිරීම්

#	Mistake වැරද්ද	Wrong වැරදි	Correct නිවැරදි	Why හේතුව
1	Forgetting the colon after if / for / while / def	<code>if x > 5</code>	<code>if x > 5:</code>	SyntaxError — the colon starts the block. Colon එක block එක අරඹයි.
2	Wrong indentation inside a block	<code>if ok: print(x)</code>	<code>if ok: print(x)</code>	Python uses indentation instead of braces. Python වරහන් වෙනුවට indentation භාවිතා කරයි.
3	Using = instead of == in a condition	<code>if mark = 40:</code>	<code>if mark == 40:</code>	= assigns; == compares. = යනු ආදේශයයි; == යනු සැසඳීමයි.
4	Adding a string to a number	<code>"5" + 5</code>	<code>int("5") + 5</code>	TypeError — convert first. පළමුව int() ට හරවන්න.
5	Dividing by a count that may be zero	<code>total / count</code>	<code>if count == 0: return 0 return total / count</code>	Guard against ZeroDivisionError. ZeroDivisionError වළක්වන්න.
6	Indexing past the end of a list	<code>marks[10] # 6 items</code>	<code>if i < len(marks): marks[i]</code>	IndexError — check the index is within range. Index එක පරාසය තුළදැයි පරීක්ෂා කරන්න.
7	Looking up a dict key that may not exist	<code>students["S099"]</code>	<code>students.get("S099") or try/except KeyError</code>	Confirm the ID exists, or use .get(). ID එක ඇතිදැයි තහවුරු කරන්න, නැතිනම් .get() යොදන්න.
8	Bare except that swallows every bug	<code>except:</code>	<code>except ValueError:</code>	Catch the specific exception; bare except hides real bugs. නිශ්චිත exception එක අල්ලන්න.
9	Averaging with len(data) when some marks are None	<code>total / len(data)</code>	<code>count only valid marks; total / count</code>	The Week 11 grade-calculator bug: skip None, count valid ones. None මග හැර වලංගු ඒවා පමණක් ගණන් කරන්න.
10	Forgetting return (function prints instead)	<code>def avg(g): print(sum(g)/len(g))</code>	<code>def avg(g): return sum(g)/len(g)</code>	Caller receives None if nothing is returned. return නොමැති නම් caller ට None ලැබේ.
11	Infinite while loop — condition never updated	<code>while n > 0: print(n)</code>	<code>while n > 0: print(n) n -= 1</code>	Update the loop variable inside the loop. ලූපය තුළ විචල්‍යය යාවත්කාලීන කරන්න.
12	Missing base case in recursion	<code>def f(n): return n + f(n-1)</code>	<code>def f(n): if n == 0: return 0 return n + f(n-1)</code>	RecursionError without a base case. Base case නොමැති නම් RecursionError.
13	Using pop() when pop(0) is needed for a queue	<code>queue.pop()</code>	<code>queue.pop(0)</code>	pop() removes the LAST item (stack); a queue serves the FIRST.

#	Mistake වැරද්ද	Wrong වැරදි	Correct නිවැරදි	Why හේතුව
				pop() අවසාන අයිතමය ඉවත් කරයි (stack); queue එකට pop(0).
14	Not closing / not using with for files	f = open("data.txt")	with open("data.txt") as f:	with closes the file automatically, even on error. with ගොනුව ස්වයංක්‍රීයව වසයි.
15	Copying a list by assignment	b = a	b = a.copy()	b = a shares one list; changes appear in both. b = a යනු එකම list එක බෙදා ගැනීමයි.

11. “Night Before the Exam” Revision Checklist

විභාගයට පෙර රාත්‍රියේ සංශෝධන ලැයිස්තුව

- ☐ 1. Recite the 3 error types with one example each (syntax / runtime / logic).
දෝෂ වර්ග 3 උදාහරණ සමඟ කියන්න.
- ☐ 2. Write the full try / except / else / finally skeleton from memory and say when each block runs.
Try/except/else/finally සැකිල්ල මතකයෙන් ලියන්න.
- ☐ 3. Match all 6 built-in exceptions to their trigger code (ValueError, TypeError, ZeroDivisionError, IndexError, KeyError, FileNotFoundError).
සාමාන්‍ය ව්‍යතිරේක 6 ඒවා ඇති කරන කේතයට ගළපන්න.
- ☐ 4. Write one raise ValueError(...) validation function (mark range 0–100).
Raise සහිත validation ශ්‍රිතයක් ලියන්න.
- ☐ 5. Read one traceback out loud: file, line, error type, message.
Traceback එකක් ශබ්ද නඟා කියවන්න.
- ☐ 6. List the 4 debugging techniques.
දෝෂ නිරාකරණ ක්‍රම 4 ලියන්න.
- ☐ 7. Recite the 6-step problem-solving process in order.
ගැටලු විසඳීමේ පියවර 6 අනුපිළිවෙලින් කියන්න.
- ☐ 8. Say which structure fits: arrival order → queue; undo → stack; sorted fast lookup → BST; survive shutdown → file.
අවශ්‍යතාවයට ගැලපෙන ව්‍යුහය කියන්න.
- ☐ 9. Write stack and queue operations using a Python list (append / pop / pop(0)).
List එකකින් stack/queue මෙහෙයුම් ලියන්න.
- ☐ 10. Write a recursive function with a clear base case (e.g., total_pages).
පැහැදිලි base case සහිත recursive ශ්‍රිතයක් ලියන්න.
- ☐ 11. Write the safe file-loading pattern: with open(...) inside try, except FileNotFoundError.
ආරක්ෂිත file-loading රටාව ලියන්න.
- ☐ 12. Explain indexing vs slicing and the excluded stop index.
Indexing/slicing සහ බැහැර වන stop index පැහැදිලි කරන්න.
- ☐ 13. Explain mutable vs immutable with b = a examples.
Mutable/immutable, b = a උදාහරණ සමඟ.
- ☐ 14. Recall the 3 case studies + which techniques each combined.
සිද්ධි අධ්‍යයන 3 සහ ඒවායේ ක්‍රම මතක් කරන්න.
- ☐ 15. State the capstone integration rule: agree on shared keys BEFORE building.
Capstone ඒකාබද්ධතා නීතිය කියන්න.
- ☐ 16. Sleep early — a rested brain debugs better than a tired one.
කලින් නිදාගන්න — විවේකී මොළය දෝෂ වඩා හොඳින් සොයයි.

12. Multiple-Choice Questions (15)

බහුවරණ ප්‍රශ්න (15)

Q1. Which error is caught before the program even runs?

- A) Logic error
- B) Runtime error
- C) Syntax error
- D) KeyError

Answer: C — Syntax errors break the grammar and stop the program from starting.

වාග් රීති දෝෂ බාවනයට පෙරම හඳුනාගනී.

Q2. A program runs without crashing but prints the wrong average. This is a:

- A) Syntax error
- B) Runtime error
- C) Logic error
- D) Traceback

Answer: C — No crash, no warning — the reasoning is wrong.

Crash නැත — නර්කයම වැරදිකි.

Q3. Which exception does `int("abc")` raise?

- A) TypeError
- B) ValueError
- C) KeyError
- D) IndexError

Answer: B — The type (str) is acceptable to `int()`, but the value cannot be converted.

වර්ගය හරි, අගය පරිවර්තනය කළ නොහැක.

Q4. Which exception does `"5" + 5` raise?

- A) ValueError
- B) TypeError
- C) ZeroDivisionError
- D) SyntaxError

Answer: B — Adding a string and an integer is an operation between wrong types.

වැරදි වර්ග අතර මෙහෙයුමකි.

Q5. `students["S099"]` where the key does not exist raises:

- A) IndexError
- B) NameError
- C) KeyError
- D) ValueError

Answer: C — Missing dictionary key → KeyError. Missing list index → IndexError.

Dict key නැති විට KeyError; list index නැති විට IndexError.

Q6. Which block always runs, whether or not an error occurred?

- A) try
- B) except
- C) else
- D) finally

Answer: D — finally runs no matter what — used for cleanup / final messages.

finally සැමවිටම බාවනය වේ.

Q7. The else block of a try statement runs:

- A) Always
- B) Only if an error occurs
- C) Only if no error occurred
- D) Before try

Answer: C — else is the success path.

else = සාර්ථක මාර්ගය.

Q8. Which structure serves items in arrival order?

- A) Stack
- B) Queue
- C) BST
- D) Dictionary

Answer: B — Queue = FIFO, first in first out.

Queue = FIFO.

Q9. Which structure suits an undo feature?

- A) Queue
- B) Linked list
- C) Stack
- D) Sequential file

Answer: C — Stack = LIFO — the last action is undone first.

Stack = LIFO.

Q10. Searching a balanced BST of 10,000 titles needs about how many comparisons?

- A) 5,000
- B) 100
- C) 14
- D) 1

Answer: C — The search space halves at every step ($\log_2 10,000 \approx 14$).

සෑම පියවරකදීම සෙවුම අඩකින් අඩු වේ.

Q11. In a Python list used as a queue, dequeue is:

- A) pop()
- B) pop(0)
- C) remove()
- D) append()

Answer: B — pop(0) removes the FIRST (oldest) item; pop() removes the last (stack).

pop(0) පළමු අයිතමය ඉවත් කරයි.

Q12. What is the essential part that stops a recursive function?

- A) The loop
- B) The base case
- C) The return type
- D) The argument

Answer: B — Without a base case, recursion never ends (RecursionError).

Base case නොමැති නම් recursion නවතින්නේ නැත.

Q13. marks[1:4] on [10,20,30,40,50] returns:

- A) [20,30,40,50]
- B) [10,20,30]
- C) [20,30,40]
- D) [30,40]

Answer: C — Slicing includes start (1) and excludes stop (4): positions 1,2,3.

Stop index ඇතුළත් නොවේ.

Q14. Which is IMMUTABLE in Python?

- A) list
- B) dict
- C) tuple
- D) set

Answer: C — Tuples (and int, float, str, bool) cannot change after creation.

Tuple සෑදූ පසු වෙනස් කළ නොහැක.

Q15. In the capstone, integrations most commonly fail because of:

- A) Slow computers
- B) Mismatched shared keys
- C) Too many functions
- D) Using Python

Answer: B — Mismatched book_id / student_id formats are the #1 reason integrations fail — agree before building.

නොගැලපෙන keys #1 හේතුවයි.

13. True / False Questions with Corrections (12)

සත්‍ය / අසත්‍ය ප්‍රශ්න සහ නිවැරදි කිරීම් (12)

Q1. A logic error always crashes the program.

Answer: FALSE — Correction: A logic error never crashes — the program runs fine but gives the wrong answer.

තර්ක දෝෂය කිසිදා crash නොවේ — වැරදි පිළිතුර දෙයි පමණි.

Q2. The except block is skipped entirely if no error occurs inside try.

Answer: TRUE

දෝෂයක් නොමැති නම් except මග හැරේ.

Q3. finally runs only when an exception is raised.

Answer: FALSE — Correction: finally always runs, error or not.

finally සැමවිටම ධාවනය වේ.

Q4. A bare except: is recommended because it catches every possible error.

Answer: FALSE — Correction: It is discouraged — it also hides bugs you didn't expect; catch specific exceptions.

හිස් except: නොකළ යුත්තකි — නොදන්නා දෝෂ ද සඟවයි.

Q5. 10 / 0 raises ValueError.

Answer: FALSE — Correction: It raises ZeroDivisionError.

එය ZeroDivisionError දෙයි.

Q6. raise can enforce rules the language itself doesn't know, like a valid mark range.

Answer: TRUE

raise මගින් ඔබේම නීති බලාත්මක කළ හැක.

Q7. A stack is first-in, first-out.

Answer: FALSE — Correction: A stack is LIFO (last-in, first-out); a queue is FIFO.

Stack = LIFO; Queue = FIFO.

Q8. In a BST, a title smaller than the root goes to the left subtree.

Answer: TRUE

කුඩා අගය වම් උප වෘක්ෂයට යයි.

Q9. A sequential file stores one record per line, read top to bottom.

Answer: TRUE

Sequential file = පේළියකට වාර්තාවක්.

Q10. marks[-1] raises an IndexError because indexes cannot be negative.

Answer: FALSE — Correction: Negative indexes count from the end — marks[-1] is the last item.

සෘණ index අගින් ගණන් කරයි — marks[-1] අවසාන අයිතමයයි.

Q11. After b = a where a is a list, appending to b also changes a.

Answer: TRUE

Assignment එකෙන් එකම list එක බෙදා ගැනේ.

Q12. print() and return do the same thing inside a function.

Answer: FALSE — Correction: print() only displays; return sends the value back to the caller and ends the function.

print() පෙන්වයි පමණි; return අගය caller ට යවයි.

14. Definitions & Short-Answer Questions (10)

අර්ථ දැක්වීම් සහ කෙටි පිළිතුරු ප්‍රශ්න (10)

Q1. Define a bug.

Answer: An error in a program that makes it crash, misbehave, or produce the wrong result.

වැඩසටහන crash කරන, වැරදි ලෙස හැසිරෙන හෝ වැරදි ප්‍රතිඵල දෙන දෝෂයකි.

Q2. Define an exception.

Answer: An event during execution that disrupts the normal flow of a program's instructions.

වැඩසටහනේ සාමාන්‍ය ගලායාම කඩාකප්පල් කරන ධාවන කාල සිදුවීමකි.

Q3. What three pieces of information does a traceback give you?

Answer: The file & line where the crash happened, the error type (exception name), and a message describing what went wrong.

File & line, error type, message.

Q4. State the purpose of the finally block.

Answer: Cleanup / guaranteed actions: it always runs whether an error occurred or not (e.g., 'Lookup attempt complete').

දෝෂය තිබුණත් නැතත් සැමවිටම ධාවනය වන අවසාන ක්‍රියා සඳහා.

Q5. Why is rubber duck debugging effective?

Answer: Explaining the code line by line forces you to slow down and re-read each step — the act of explaining reveals the bug.

පේළියෙන් පේළිය පැහැදිලි කිරීමේදීම දෝෂය හෙළි වේ.

Q6. Why choose a structure over a plain array of numbers for student records?

Answer: A plain array loses the link between a student and their own grades — structures keep related fields (id, name, grades) together.

සරල array එකෙන් ශිෂ්‍යයා-ලකුණු සම්බන්ධය නැති වේ; structure එක ඒවා එකට තබයි.

Q7. Explain FIFO and LIFO with one real use each.

Answer: FIFO (queue): jobs print in arrival order. LIFO (stack): the last submitted job is cancelled first (undo).

FIFO (queue): පැමිණි අනුපිළිවෙලින් මුද්‍රණය. LIFO (stack): අවසාන ක්‍රියාව මුලින්ම අහෝසි.

Q8. Why does load_catalog() catch FileNotFoundError?

Answer: So a missing file starts an empty catalog instead of crashing the whole program.

File එක නැති විට crash නොවී හිස් catalog එකක් ආරම්භ කිරීමට.

Q9. What is a shared data contract in the capstone?

Answer: The agreed book_id / student_id key formats that every module reads and writes — it is what makes integration possible.

සෑම module එකක්ම භාවිතා කරන එකඟ වූ key ආකෘතිය — ඒකාබද්ධතාවය හැකි කරන්නේ එයයි.

Q10. Name the two parts every recursive function must have.

Answer: A base case (where it stops) and a recursive case (where it calls itself on smaller input).

Base case සහ recursive case.

15. Comparison Questions (6)

සංසන්දන ප්‍රශ්න (6)

Q1. Compare a syntax error and a runtime error, giving one example of each.

Answer: A syntax error is broken grammar caught before the program runs (e.g., a missing colon); a runtime error occurs while running when valid code meets an impossible situation (e.g., $10/0 \rightarrow \text{ZeroDivisionError}$). A syntax error prevents any execution; a runtime error can be caught with try/except so the program continues.

Syntax දෝෂය ධාවනයට පෙර හඳුනාගැනීම (colon නැතිවීම); runtime දෝෂය ධාවනය අතරතුර ඇතිවේ ($10/0$). Runtime දෝෂ try/except වලින් ඇල්ලිය හැක.

Q2. Compare a for loop and a while loop and state when each is preferred.

Answer: A for loop iterates over a known sequence (list, range, file lines) and ends when it is exhausted — preferred when the number of repetitions is known. A while loop repeats while a condition stays true — preferred when the stopping point depends on runtime events (e.g., valid input received). A while loop risks infinite looping if the condition is never updated.

For: පුනරාවර්තන ගණන දන්නා විට. While: කොන්දේසියක් මත — නොදන්නා විට; infinite loop අවදානම ඇත.

Q3. Compare print() and return inside a function.

Answer: print() only displays a value on screen and the function continues; return sends the value back to the caller (so it can be stored: $x = f()$) and immediately ends the function. A function that only prints returns None to its caller.

print() පෙන්වයි; return caller ට යවා ශ්‍රිතය අවසන් කරයි. Print පමණක් කරන ශ්‍රිතයක් None ලබා දෙයි.

Q4. Compare a queue and a stack using the print-lab case study.

Answer: The print queue is FIFO — jobs print in the order they arrive (append to the end, pop(0) from the front). The undo stack is LIFO — cancel_last_job() pops the most recently submitted job. submit_job() feeds both at once: the same event, two different views of it.

Queue = FIFO (මුද්‍රණ අනුපිළිවෙල); Stack = LIFO (undo). submit_job() දෙකටම එකවර දන්නා දෙයකි.

Q5. Compare a BST and a linked list for searching, using the library case study.

Answer: A balanced BST halves the search space at each comparison — about 14 steps for 10,000 titles — but requires a sorted key (title). A linked list is searched linearly (up to n steps) but suits data with no useful ordering, such as browsing by author. That is why search_by_title uses the BST and search_by_author uses a linked list.

BST: වර්ග කළ key එකකින් log-time සෙවීම. Linked list: රේඛීය සෙවීම — අනුපිළිවෙලක් නැති දත්තවලට සුදුසුයි.

Q6. Compare mutable and immutable objects and explain the b = a effect.

Answer: Mutable objects (list, dict) can change in place, so after $b = a$ both names share one object — $b.append(9)$ also changes a . Immutable objects (int, str, tuple) cannot change; $b = b + 1$ creates a new object and a is untouched. Use $a.copy()$ to get an independent list.

Mutable: $b = a$ පසු දෙකම එකම object එකයි. Immutable: වෙනස්කම් නව object සාදයි. ස්වාධීන පිටපතකට copy().

16. Code-Output Questions (8)

කේත ප්‍රතිදාන ප්‍රශ්න (8)

Q1. What is the output?

```
marks = [72, 65, 80, 91]
print(marks[-1])
print(marks[1:3])
```

Answer: 91 then [65, 80]

-1 is the last item; slice [1:3] includes positions 1 and 2, excluding 3. | -1 = අවසාන අයිතමය; slice එකේ stop බැහැර වේ.

Q2. What is the output?

```
try:
    x = int("12")
except ValueError:
    print("Bad")
else:
    print("Good", x)
finally:
    print("Done")
```

Answer: Good 12 then Done

"12" converts successfully, so except is skipped, else runs, finally always runs. | සාර්ථක නිසා else ධාවනය වේ; finally සැමවිටම.

Q3. What is the output?

```
try:
    x = int("abc")
    print("Converted")
except ValueError:
    print("Bad input")
finally:
    print("Done")
```

Answer: Bad input then Done

int("abc") raises ValueError before "Converted" prints; except handles it; finally still runs. | දෝෂය නිසා try හි ඉතිරිය මඟ හැරේ.

Q4. What is the output?

```
def f(n):
    if n == 0:
        return 0
    return n + f(n - 1)
print(f(4))
```

Answer: 10

$4 + 3 + 2 + 1 + 0 = 10$ — base case $n == 0$ stops the recursion. | $4+3+2+1+0 = 10$; base case එකෙන් නවතී.

Q5. What is the output?

```
q = []
q.append("A"); q.append("B"); q.append("C")
print(q.pop(0))
print(q.pop())
```

Answer: A then C

pop(0) dequeues the first (queue behaviour); pop() pops the last (stack behaviour). | pop(0) = queue; pop() = stack.

Q6. What is the output?

```
a = [1, 2]
b = a
b.append(3)
```

```
print(a)
```

Answer: [1, 2, 3]

`b = a` shares the same list — appending through `b` changes `a` too. | `b = a` යනු එකම list එකයි.

Q7. What is the output?

```
def get_mark(d, sid):
    try:
        return d[sid]
    except KeyError:
        return "ID not found"

print(get_mark({"S001": 78}, "S009"))
```

Answer: ID not found

"S009" is missing, so the lookup raises `KeyError`, which the `except` converts to a friendly message. | නැති key එක `KeyError` දෙයි; `except` එය මිත්‍රශීලී පණිවිඩයක් කරයි.

Q8. What is the output?

```
batch = {"sub_jobs": [
    {"pages": 5},
    {"sub_jobs": [{"pages": 3}, {"pages": 8}]}
]}

def total_pages(job):
    if "sub_jobs" not in job:
        return job["pages"]
    return sum(total_pages(j) for j in job["sub_jobs"])

print(total_pages(batch))
```

Answer: 16

$5 + (3 + 8) = 16$ — recursion totals pages at any nesting depth. | $5 + (3+8) = 16$; recursion ඕනෑම ගැඹුරකට ගණන් කරයි.

17. Error-Identification & Debugging Questions (6)

දෝෂ හඳුනාගැනීම් සහ නිරාකරණ ප්‍රශ්න (6)

Q1. Name the error type and fix it.

```
if mark > 40
    print("Pass")
```

Answer: Syntax error — missing colon. Fix: if mark > 40:

වාග් රීති දෝෂය — colon නැත.

Q2. This crashes on the given data. Name the exception and fix the function.

```
students = {"S001": 78, "S003": None}
def average_marks(data):
    total = 0
    for mark in data.values():
        total += mark
    return total / len(data)
```

Answer: TypeError — adding None to total. Fix: skip None marks and divide by a count of valid marks only (if mark is not None: total += mark; count += 1; return total / count).

None එකතු කිරීම TypeError දෙයි. None මග හැර වලංගු ලකුණු ගණනින් බෙදන්න.

Q3. Identify the exception raised and the safest one-line prevention.

```
marks = [70, 80, 90]
print(marks[5])
```

Answer: IndexError — index 5 on a 3-item list. Prevention: check if 5 < len(marks) first (or catch IndexError).

IndexError — index එක len(marks) ට වඩා කුඩාදැයි පරීක්ෂා කරන්න.

Q4. The program never ends. Name the error category and fix it.

```
n = 5
while n > 0:
    print(n)
```

Answer: Logic error (infinite loop) — the condition never changes. Fix: add n -= 1 inside the loop.

තර්ක දෝෂය (අනන්ත ලූපය) — ලූපය තුළ n -= 1 එකතු කරන්න.

Q5. What is wrong with this exception handler?

```
try:
    process()
except:
    pass
```

Answer: A bare except that silently swallows every error — including unexpected bugs. Fix: catch the specific exception(s) and at least log/print the error.

හිස් except සියලු දෝෂ නිශ්ශබ්දව ගිලිසී — නිශ්චිත exception එක අල්ලා දෝෂය වාර්තා කරන්න.

Q6. Read the traceback. State the file, line, error type and a fix.

```
Traceback (most recent call last):
  File "grades.py", line 12, in <module>
    average = total / count
ZeroDivisionError: division by zero
```

Answer: File grades.py, line 12; ZeroDivisionError; count is 0. Fix: guard with if count == 0: return 0 (or catch ZeroDivisionError).

grades.py, ශ්‍රේණිය 12; ZeroDivisionError; count == 0 විට 0 ලබා දෙන්න.

18. Algorithm / Pseudocode Questions (4)

ඇල්ගොරිතම / ව්‍යාජ කේත ප්‍රශ්න (4)

Q1. Write pseudocode to compute the average of a list of grades, safely handling an empty list.

හිස් ලැයිස්තුවක් ආරක්ෂිතව හසුරුවමින් ලකුණු ලැයිස්තුවක සාමාන්‍යය ගණනය කිරීමට ව්‍යාජ කේතය ලියන්න.

Model answer:

```
BEGIN average(grades)
  IF length of grades = 0 THEN
    RETURN 0
  ENDIF
  total ← 0
  FOR each mark IN grades DO
    total ← total + mark
  ENDFOR
  RETURN total / length of grades
END
```

Empty-list guard පළමුව — ZeroDivisionError වළක්වයි.

Q2. Write pseudocode that reads marks until the user types -1, then prints how many were at risk (below 40).

-1 ලැබෙන තුරු ලකුණු කියවා 40 ට අඩු ගණන මුද්‍රණය කරන ව්‍යාජ කේතය ලියන්න.

Model answer:

```
BEGIN
  atRisk ← 0
  INPUT mark
  WHILE mark ≠ -1 DO
    IF mark < 40 THEN
      atRisk ← atRisk + 1
    ENDIF
    INPUT mark
  ENDWHILE
  OUTPUT atRisk
END
```

නොදන්නා ගණනක් — while ලූපයකි (sentinel = -1).

Q3. Give the six problem-solving steps you would follow before and while building a grade tracker.

Grade tracker එකක් සෑදීමට පෙර/අතරතුර අනුගමනය කරන පියවර හය ලියන්න.

Model answer:

1. Understand - what is being asked?
2. Decompose - store grades, compute average, flag < 40
3. Choose structures - list of records (structures)
4. Design algorithm - pseudocode / flowchart
5. Implement - write the Python
6. Test & refine - run it and fix what breaks

Week 12 හි worked example එකම වේ.

Q4. Write pseudocode for searching a BST by title, counting comparisons.

සැසඳීම් ගණන් කරමින් BST එකක title එකකින් සෙවීමට ව්‍යාජ කේතය ලියන්න.

Model answer:

```
FUNCTION search(node, title, steps)
  IF node = NULL THEN RETURN (NOT FOUND, steps)
  steps ← steps + 1
  IF title = node.title THEN RETURN (node, steps)
  IF title < node.title THEN
    RETURN search(node.left, title, steps)
  ELSE
    RETURN search(node.right, title, steps)
END FUNCTION
```

කුඩා → වමට; විශාල → දකුණට; සෑම පියවරකදීම steps + 1.

19. Flowchart Questions (3)

ගැලීම් සටහන් ප්‍රශ්න (3)

Standard symbols: oval = Start/End, parallelogram = Input/Output, rectangle = Process, diamond = Decision, arrows = flow.

සම්මත සංකේත: ඕවලාකාරය = ආරම්භය/අවසානය, සමාන්තරාස්‍රය = ආදාන/ප්‍රතිදාන, සෘජුකෝණාස්‍රය = ක්‍රියාවලිය, දියමන්තිය = තීරණය.

Q1. Draw a flowchart that inputs a mark and prints "AT RISK" if it is below 40, otherwise "OK".

ලකුණක් ලබාගෙන 40 ට අඩු නම් "AT RISK", නැතිනම් "OK" මුද්‍රණය කරන ගැලීම් සටහන අඳින්න.

Model answer (describe/draw):

```
(Start) -> [/Input mark/] -> <mark < 40 ?>
  Yes -> [/Print "AT RISK"/] -> (End)
  No  -> [/Print "OK"/] -> (End)
```

එක diamond එකක් — if-else ආකෘතිය.

Q2. Draw a flowchart for the Write → Test → Debug cycle from Week 12.

Week 12 හි Write → Test → Debug චක්‍රයේ ගැලීම් සටහන අඳින්න.

Model answer (describe/draw):

```
(Start) -> [Write code] -> [Run & test] -> <Hit an error?>
  Yes -> [Debug & fix] -> back to [Run & test]
  No  -> (End: program behaves correctly)
```

දෝෂ නැති වන තුරු පුනරාවර්තනය වේ.

Q3. Draw a flowchart that keeps asking for a mark until a valid number is entered (use the try/except idea as a decision).

වලංගු සංඛ්‍යාවක් ලැබෙන තුරු ලකුණක් ඉල්ලන ගැලීම් සටහන අඳින්න.

Model answer (describe/draw):

```
(Start) -> [/Input text/] -> [Try: convert to int]
  -> <Conversion OK?>
    No -> [/Print "Please enter a number"/] -> back to [/Input text/]
    Yes -> [/Print "Mark recorded"/] -> (End)
```

ValueError → නැවත ඉල්ලීම; සාර්ථක → අවසන්.

20. Short Programming Questions (5)

කෙටි ක්‍රමලේඛන ප්‍රශ්න (5)

Q1. Write `set_mark(students, student_id, mark)` that raises `ValueError` unless $0 \leq \text{mark} \leq 100$.

$0 \leq \text{mark} \leq 100$ නොවේ නම් `ValueError` raise කරන ශ්‍රිතය ලියන්න.

Model answer:

```
def set_mark(students, student_id, mark):
    if not 0 <= mark <= 100:
        raise ValueError("Mark must be between 0 and 100")
    students[student_id] = mark
```

Q2. Write `get_mark(students, student_id)` that returns the mark, "ID not found" for a missing ID, and "No mark recorded" when the stored mark is None.

ලකුණ, නැති ID එකකට "ID not found", None නම් "No mark recorded" ලබා දෙන ශ්‍රිතය ලියන්න.

Model answer:

```
def get_mark(students, student_id):
    try:
        mark = students[student_id]
    except KeyError:
        return "ID not found"
    else:
        return mark if mark is not None else "No mark recorded"
```

Week 11 'Safe Marks Lookup' practical එකේ ආකෘතියයි.

Q3. Write `highest_scorer(students)` returning the record with the top average (each record has a "grades" list). Skip bad records safely.

ඉහළම සාමාන්‍යය ඇති වාර්තාව ලබා දෙන ශ්‍රිතය ලියන්න; නරක වාර්තා ආරක්ෂිතව මඟ හරින්න.

Model answer:

```
def highest_scorer(students):
    best, best_avg = None, -1
    for s in students:
        try:
            avg = sum(s["grades"]) / len(s["grades"])
        except (KeyError, TypeError, ZeroDivisionError):
            continue # skip bad record
        if avg > best_avg:
            best, best_avg = s, avg
    return best
```

Week 12 Case Study 1 practical extension එකයි.

Q4. Write `safe_cancel()` so that cancelling on an empty undo stack prints a friendly message instead of an error.

හිස් undo stack එකක cancel කිරීමේදී දෝෂයක් වෙනුවට මිත්‍රශීලී පණිවිඩයක් දෙන ලෙස ලියන්න.

Model answer:

```
def safe_cancel():
    if not undo_stack:
        print("Nothing to cancel.")
        return None
    job = undo_stack.pop()
    print_queue.remove(job)
    return job
```

Week 12 Case Study 3 practical extension එකයි.

Q5. Write `append_borrow(book_id, student_id)` that appends one line "`book_id|student_id`" to `borrow_log.txt`.

`borrow_log.txt` ට "`book_id|student_id`" ඡේදයක් එකතු කරන ශ්‍රිතය ලියන්න.

Model answer:

```
def append_borrow(book_id, student_id):
    with open("borrow_log.txt", "a") as f:
        f.write(f"{book_id}|{student_id}\n")
```

Capstone Team 1 ଖି sequential-file ଠପାଉଛି.

21. Substantial Practical Programming Problem

විශාල ප්‍රායෝගික ක්‍රමලේඛන ගැටලුව

Mini Library System — Borrow, Waitlist & Report | කුඩා පුස්තකාල පද්ධතිය — Borrow, Waitlist සහ Report

Task (30–40 marks style): Build a small library manager that combines structures, functions, queues, files, and exception handling — the exact Week 12 pattern.

කාර්යය: structures, functions, queues, files සහ exception handling ඒකාබද්ධ කරන කුඩා පුස්තකාල කළමනාකරුවෙකු ගොඩනගන්න — Week 12 රටාවම.

- (a) Store books as a list of structures: {"book_id", "title", "on_loan"}. (4 marks)
පොත් list of structures ලෙස ගබඩා කරන්න.
- (b) borrow(book_id, student_id): raise ValueError if the book is already on loan; otherwise mark it on loan and append "book_id|student_id" to borrow_log.txt. (8 marks)
දැනටමත් ණයට ගෙන ඇත්නම් ValueError; නැතිනම් සලකුණු කර log ගොනුවට ලියන්න.
- (c) join_waitlist(book_id, student_id) and notify_next(book_id) using a FIFO queue per book. (8 marks)
පොතකට FIFO පෝලීමක් භාවිතා කරන්න.
- (d) return_book(book_id): free the book and notify the next waiting student, if any. (6 marks)
පොත නිදහස් කර ඊළඟ ශිෂ්‍යයාට දන්වන්න.
- (e) load_log(): rebuild loan states from borrow_log.txt at startup, starting empty if the file is missing. (6 marks)
ආරම්භයේදී log එකෙන් තත්ත්වය නැවත ගොඩනගන්න; ගොනුව නැතිනම් හිස්ව පරිභවනය.
- (f) A demo run showing one successful borrow, one rejected borrow, a waitlist join, and a return that notifies. (4 marks)
සාර්ථක borrow, ප්‍රතික්ෂේපිත borrow, waitlist join සහ notify කරන return එකක් පෙන්වන demo ධාවනයක්.

Model solution:

```
books = [
    {"book_id": "B001", "title": "Dune", "on_loan": False},
    {"book_id": "B002", "title": "Moby Dick", "on_loan": False},
]
waitlists = {} # book_id -> queue (list)

def find_book(book_id):
    for b in books:
        if b["book_id"] == book_id:
            return b
    raise KeyError(f"Unknown book: {book_id}")

def borrow(book_id, student_id):
    book = find_book(book_id)
    if book["on_loan"]:
        raise ValueError(f"{book['title']} is already on loan")
    book["on_loan"] = True
    with open("borrow_log.txt", "a") as f:
        f.write(f"{book_id}|{student_id}\n")

def join_waitlist(book_id, student_id):
    waitlists.setdefault(book_id, []).append(student_id) # enqueue

def notify_next(book_id):
    q = waitlists.get(book_id, [])
    if q:
        return q.pop() # dequeue (FIFO)
    return None

def return_book(book_id):
    book = find_book(book_id)
    book["on_loan"] = False
```

```

    nxt = notify_next(book_id)
    if nxt:
        print(f"Notify {nxt}: {book['title']} is now available")

def load_log():
    try:
        with open("borrow_log.txt", "r") as f:
            for line in f:
                book_id, student_id = line.strip().split("|")
                try:
                    find_book(book_id)["on_loan"] = True
                except KeyError:
                    print(f"Skipped unknown book {book_id}")
    except FileNotFoundError:
        print("No log file - starting empty")

# ---- demo run ----
load_log()
try:
    borrow("B001", "S001")          # success
    borrow("B001", "S002")          # rejected -> ValueError
except ValueError as e:
    print("Rejected:", e)
    join_waitlist("B001", "S002")   # S002 waits
return_book("B001")                 # frees B001, notifies S002

```

Marking pattern to quote: chose the right structures (list of dicts + FIFO queue), wrote small functions, and wrapped every risky step (unknown book, double borrow, missing file) in exceptions — data persists via a sequential log file so the next run picks up where you left off.

සඳහන් කළ යුතු රටාව: නිවැරදි ව්‍යුහ තෝරා ගැනීම, කුඩා ශ්‍රිත ලිවීම, සෑම අවදානම් පියවරක්ම exceptions වලින් ඔතා තිබීම, sequential log ගොනුවෙන් දත්ත රැඳී පැවතීම.